

La Couche de Transport dans le modèle de l'IETF (partie 2 : compléments)

Thierry DESPRATS

Université de Toulouse

FSI – Dpt Info // IRIT- SIERA

Thierry.Desprats@irit.fr

Agenda

- TCP : Compléments
 - Contrôle de flux
 - Gestion des retransmissions
 - Contrôle de congestion
 - RFCs

- MPTCP
 - Généralités
 - Fonctionnement

- SCTP
 - Caractéristiques
 - Gestion des associations
 - Transfert de données
 - Protocole
 - RFCs

TCP (1/31) Contrôle de flux (1/3)

□ Fenêtre fermée :

□ Principe :

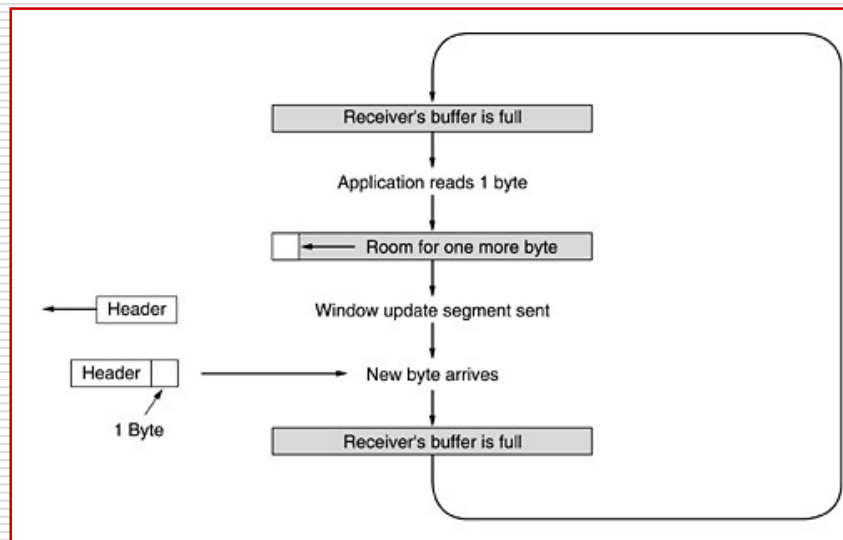
- Quand le buffer du récepteur est plein, le récepteur ordonne l'arrêt de l'émission en positionnant un $W=0$
= = > Fermeture de la fenêtre
- L'émetteur n'a le droit de transmettre que :
 - Des données urgentes (bit **URG** positionné)
 - Des paquets « **window probe** » :
 - Segments de un octet non significatif
(le numéro d'acquittement reçu n'est pas incrémenté)
 - Le premier envoi se fait 5 secondes après la réception d'un $W=0$
 - La durée entre deux tests double jusqu'à atteindre la minute
- Côté récepteur : quand l'application libère les ressources, un $W=x$ est envoyé pour inviter à la reprise du transfert.

TCP (2/31) Contrôle de flux (2/3)

□ Syndrome de la fenêtre stupide :

- SWS – (Silly Window Syndrome)
- Problème :
 - Vitesse de vidage du buffer du récepteur très lente (e.g. lecture octet par octet)
 - Alternance $W=0$, $W=x$, $W=0$, $W=x$ et x très petit
 - Equilibre trouvé : l'émetteur n'envoie que de petits segments
= = > Réseau sous-utilisé !

□ Illustration :



Extrait de A. S. Tanenbaum

TCP (3/31) Contrôle de flux (3/3)

□ Syndrome de la fenêtre stupide :

□ Solution (Clark) :

■ Côté récepteur :

- Annonce de la réouverture de la fenêtre que lorsqu'une taille suffisante est disponible
- Ne pas annoncer de petites fenêtres !

RBS //capacité du buffer de réception associé à cette connexion

MSS //valeur retenue pour la MSS

RWS //taille de la fenêtre de réception

CalculW :

if RWS $\geq$ Min(MSS, RBS/2) then

W $\leftarrow$ RWS; // réouverture de la fenêtre

else W $\leftarrow$ 0; // fermeture maintenue

return W;

■ Côté émetteur :

- Groupage : attendre MSS avant d'envoyer un segment
- Ne pas transmettre de petits segments !

TCP (4/31) Gestion des retransmissions (1/4)

□ Gestion des temporisations :

□ **RTO** : Retransmission Time Out

□ Problème :

- Trouver les valeurs des RTOs de façon à ne pas dégrader les performances :
 - Si RTO trop court : retransmissions inutiles
 - Si RTO trop long : attente trop importante entre deux retransmissions
- Les délais d'acheminement varient en fonction des réseaux traversés :
 - débits
 - charge
 - stabilité des routes

TCP (5/31) Gestion des retransmissions (2/4)

□ Gestion des temporisations :

□ Principe :

- Détermination et ajustement dynamique du RTO d'une connexion :
= = > *Variations de l'état du réseau intégrées*
- Mécanismes :
 - Mesure permanente du **RTT** (Round Trip Time) : temps écoulé entre l'émission d'un segment et la réception de son acquittement [RFC 791]
 - Mesure du RTT : Ne pas faire de mesure dans le cas d'un segment retransmis [Algorithme de Karn]
 - Mais mesurer également la **variation** du RTT [Van Jacobson 1988] (TCP ne s'adaptait pas aux montées de charge)
 - Calculer le **RTO** en se basant sur ces variables et des facteurs correctifs dans le cas normal, le doubler à chaque prochaine retransmission sinon.

TCP (6/31) Gestion des retransmissions (3/4)

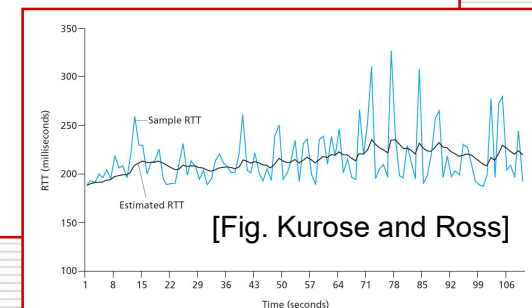
□ Gestion des temporisations :

□ Algorithme [RFC 6298] :

```
RTT-Mesuré      // Dernier RTT mesuré
RTT-Moy         // Temps RTT moyen (moyenne mobile)
DEV-RTT         // Déviation moyenne
alpha           // Facteur correctif habituellement à 1/8
beta            // Facteur correctif habituellement fixé à 1/4
RTO             // Temporisation de retransmission (init à 1s)

if réception ACK ne portant pas sur une retransmission then
  // Mise à jour des variables liées au RTT
  RTT-Moy  $\leftarrow$  (1 - alpha) * RTT-Moy + alpha * RTT-Mesuré;
  DEV-RTT  $\leftarrow$  (1 - Beta) * DEV-RTT + Beta * |RTT-Mesuré - RTT-Moy|;
  // Mise à jour du RTO
  RTO  $\leftarrow$  RTT-Moy + (4 * DEV-RTT);

if RTO écoulé et retransmission then
  RTO  $\leftarrow$  2 * RTO;
```



TCP (7/31) Gestion des retransmissions (4/4)

□ Retransmission rapide :

□ Fast Retransmit (FR)

□ Problème :

- RTO souvent surdimensionné

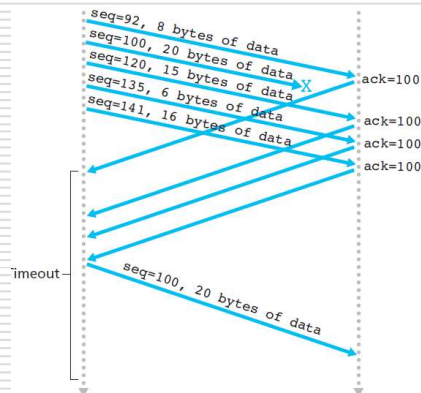
= = > *Interruption des transmissions, transfert moins performant !*

□ Principe :

- Anticiper la retransmission :

Après la réception de 3 duplicatas de segments portant un ACK=x, on retransmet immédiatement le segment concerné sans attendre l'expiration du RTO associé.

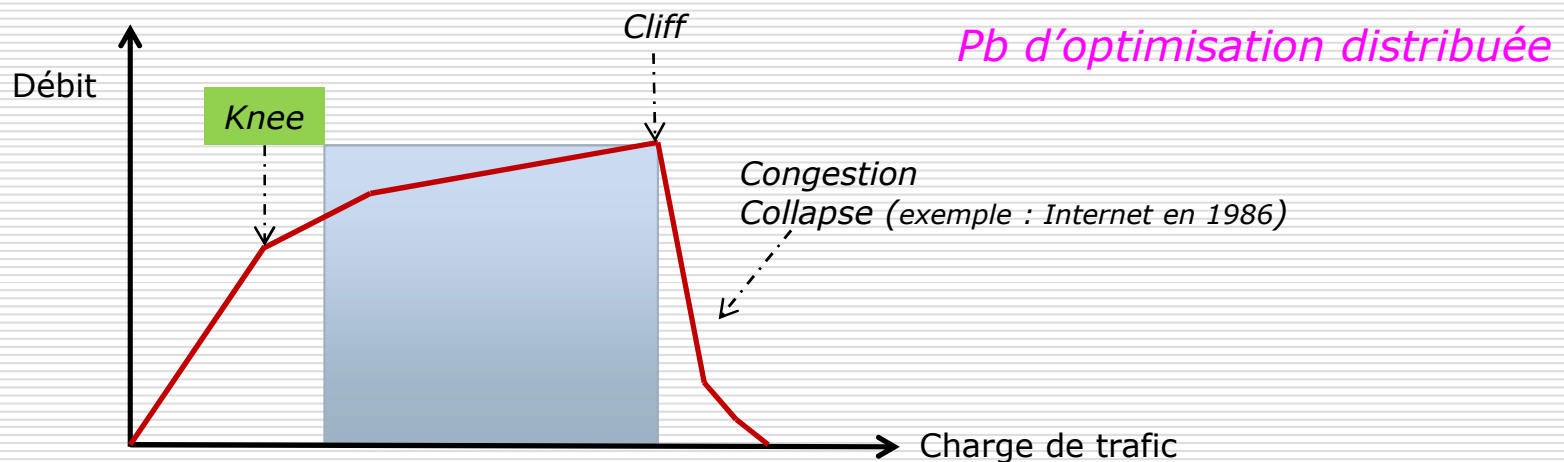
(On considère que la réception de 3 ACK dupliqués est un indicateur fiable d'une perte de paquet eu égard à la probabilité négligeable de déséquencelement et de duplication des paquets dans le réseau IP)



TCP (8/31) Contrôle de congestion (1/23)

□ Bases du contrôle de congestion :

- Problème : de multiples agents distribués tentent d'utiliser des ressources partagées (dans notre cas les réseaux à commutation de paquets)
- Objectif : fournir une bonne utilisation des ressources et en même temps une performance acceptable pour les utilisateurs du réseau.
- Formalisation :



TCP (9/31) Contrôle de congestion (2/23)

□ Fenêtre de congestion :

□ Principe général :

- Côté Emetteur autogérer une fenêtre de congestion
 C_WND

- Donc : $SND_WND = \text{Min}(R_WND, C_WND)$

□ Algorithmes de régulation :

- TCP [RFC 793] sans contrôle de congestion

- **Stratégies réactives**

- Tahoe (SS + CA)
- Reno (FR)
- New Reno
- SACK

- **Stratégies prédictives, impliquant les extrémités**

- Vegas

- **Stratégies impliquant les routeurs**

- ERD
- ECN

TCP (10/31) Contrôle de congestion (3/23)

□ Objectif :

- Pour chaque source, contrôler le débit d'émission en fonction de la capacité des réseaux sous-jacents (exploiter l'information de *feedback*)

□ Détection d'une congestion :

- Soit par l'allongement des temps de transit (les acquittements sont retardés)
- Soit par des pertes de données (les acquittements n'arrivent pas)
= = > Réseau surchargé

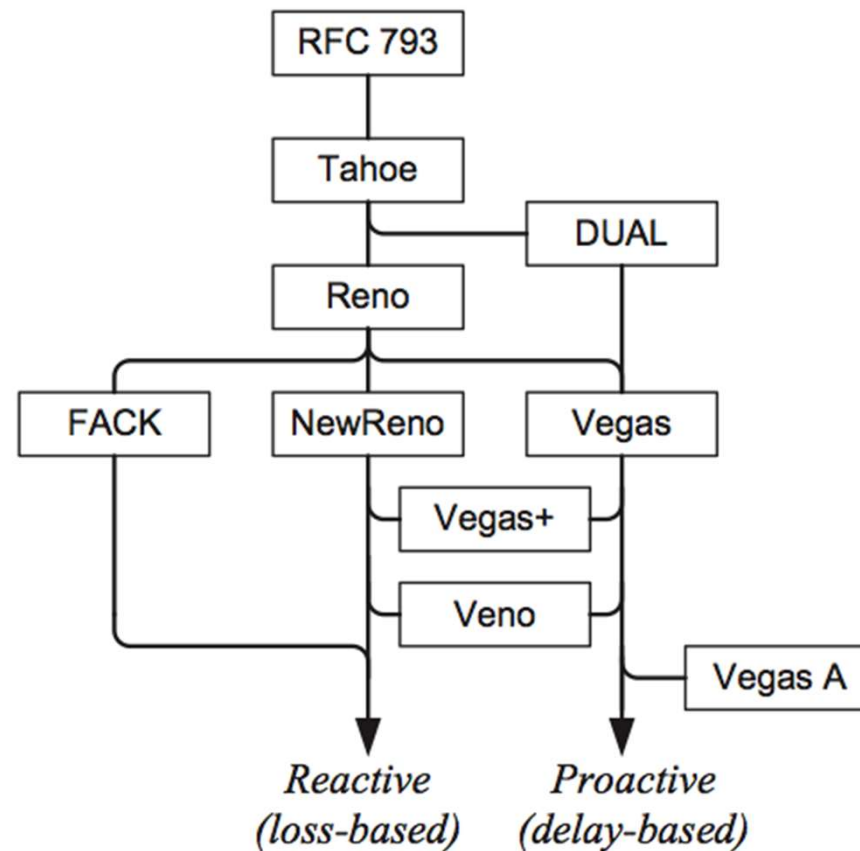
□ Problème :

- Eviter que TCP par ses réactions n'amplifie la congestion et donc accentue la perte de paquets dans le réseau
- Déterminer la capacité du réseau
= = > Adaptation à la topologie et à la charge !

TCP (11/31) Contrôle de congestion (4/23)

□ Historique :

- Stratégies originelles n'impliquant que les extrémités :



TCP (12/31) Contrôle de congestion (5/23)

□ Historique :

TCP Variant	Section	Year	Base	Added/Changed Modes or Features	Mod ¹	Status	Implementation			
							BSD ²	Linux	Win	Mac
<i>TCP Tahoe</i> [14]	II-A	1988	RFC793	Slow Start, Congestion Avoidance, Fast Retransmit	S	Obsolete Standard	>4.3	1.0		
<i>TCP-DUAL</i> [15]	II-B	1992	Tahoe	Queuing delay as a supplemental congestion prediction parameter for Congestion Avoidance	S	Experimental				
<i>TCP Reno</i> [16], [17]	II-C	1990	Tahoe	Fast Recovery	S	Standard	>4.3 >F2.2	> 1.3.90	>95/NT	
<i>TCP NewReno</i> [18], [19]	II-D	1999	Reno	Fast Recovery resistant to multiple losses	S	Standard	>F4	> 2.1.36		>10.4.6 (opt)
<i>TCP SACK</i> [20]	II-E	1996	RFC793	Extended information in feedback messages	P+S+R	Standard	>S2.6, >N1.1, >F2.1R	> 2.1.90	> 98	> 10.4.6
<i>TCP FACK</i> [21]	II-F	1996	Reno, SACK	SACK-based loss recovery algorithm	S	Experimental	>N1.1	>2.1.92		
<i>TCP-Vegas</i> [22]	II-G	1995	Reno	Bottleneck buffer utilization as a primary feedback for the Congestion Avoidance and secondary for the Slow Start	S	Experimental		> 2.2.10		
<i>TCP-Vegas+</i> [23]	II-H	2000	NewReno, Vegas	Reno/Vegas Congestion Avoidance mode switching based of RTT dynamics	S	Experimental				
<i>TCP-Veno</i> [24]	II-I	2002	NewReno, Vegas	Reno-type Congestion Avoidance and Fast Recovery increase/decrease coefficient adaptation based on bottleneck buffer state estimation	S	Experimental		> 2.6.18		
<i>TCP-Vegas A</i> [25]	II-J	2005	Vegas	Adaptive bottleneck buffer state aware Congestion Avoidance	S	Experimental				

¹ TCP specification modification: S = the sender reactions, R = the receiver reactions, P = the protocol specification

² S for Sun, F for FreeBSD, N for NetBSD

TCP (13/31) Contrôle de congestion (6/23)

□ Algorithme phase « Slow Start » (MI) :

CWND //fenêtre de congestion exprimée en MSS

Initialement :

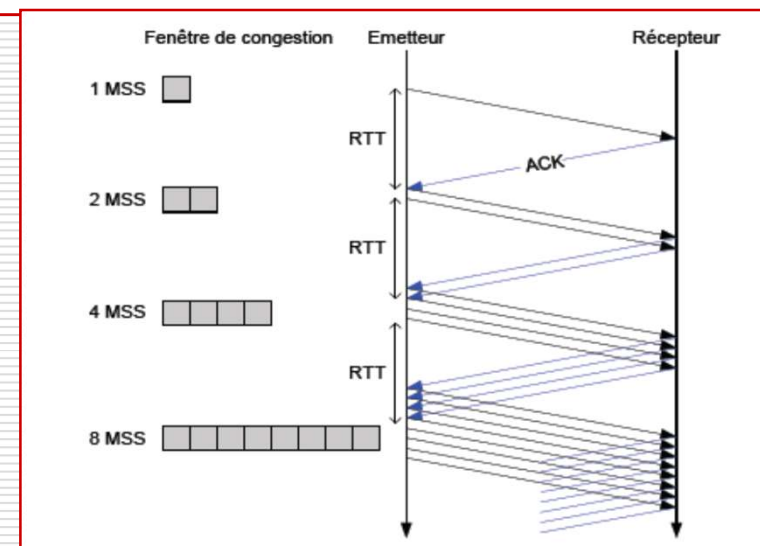
$CWND \leftarrow 1;$ // démarrage lent

A chaque ACK individuel non dupliqué reçu :

$CWND \leftarrow CWND + 1;$ // augmentation exponentielle rapide

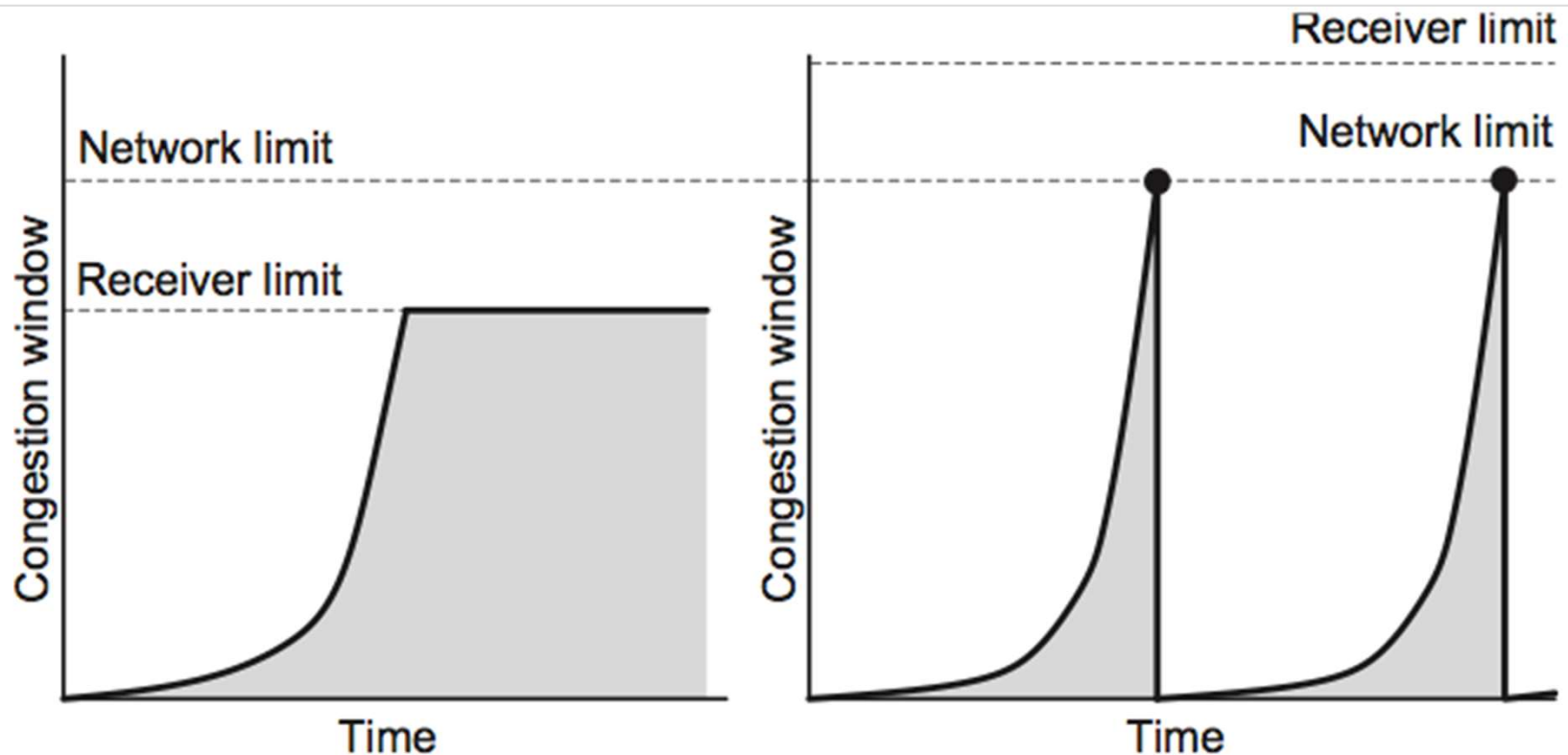
// $CWND \leftarrow CWND * 2$ à chaque groupe d'ACK (RTT)

- Eviter la surcharge d'un lien lent dès le départ
- Exploiter la capacité de liens rapides



TCP (14/31) Contrôle de congestion (7/23)

- Algorithme phase « Slow Start » (MI) :
 - Illustration efficacité SS :



TCP (15/31) Contrôle de congestion (8/23)

□ Algorithme phase « Congestion Avoidance » (AIMD) :

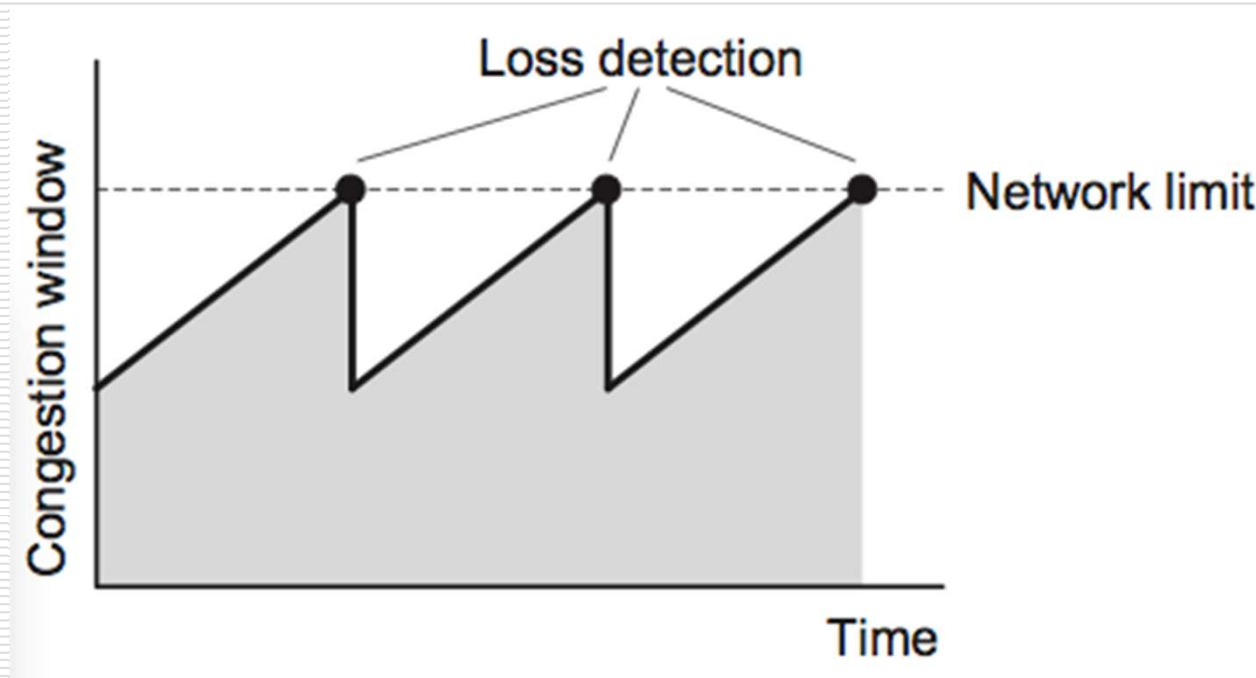
■ Algorithme :

```
CWND    //fenêtre de congestion exprimée en MSS
init     //valeur minimale seuil de la fenêtre de congestion

Initialement :
    CWND ← init;
A chaque ACK reçu (groupe, donc à chaque RTT) :
    CWND ← CWND + 1; // augmentation linéaire (AI)
if retransmission constatée then
    CWND ← init;    // réduction multiplicative (MD)
```

TCP (16/31) Contrôle de congestion (9/23)

- Algorithme phase
« Congestion Avoidance » (AIMD) :
 - Illustration efficacité CA :



TCP (17/31) Contrôle de congestion (10/23)

□ TCP Tahoe :

- Combiner (SS+CA) et intégration d'un seuil
- Variation du seuil
 - à chaque retransmission le seuil se voit affecté d'une valeur égale à la moitié de CWND et celle-ci repasse à 1.

```
CWND      //fenêtre de congestion exprimée en MSS
SSTresh   //seuil d'évitement de congestion (init à 64kB)

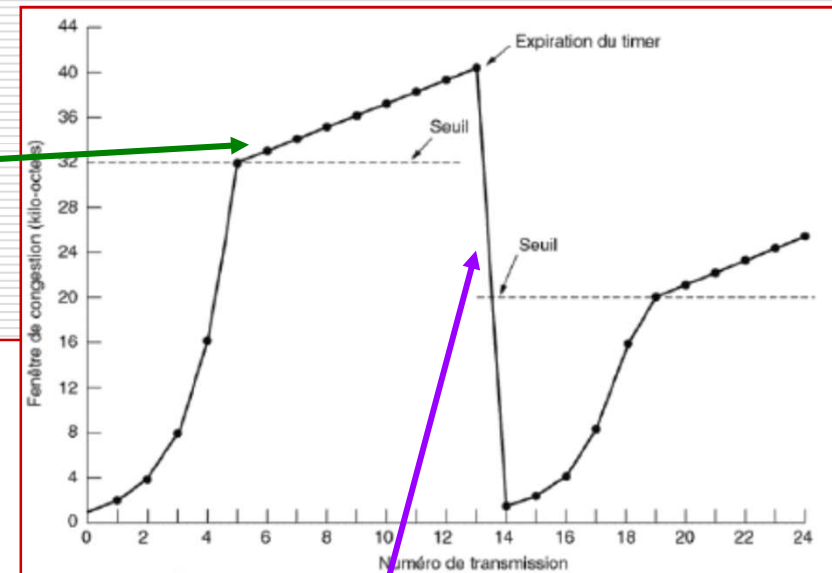
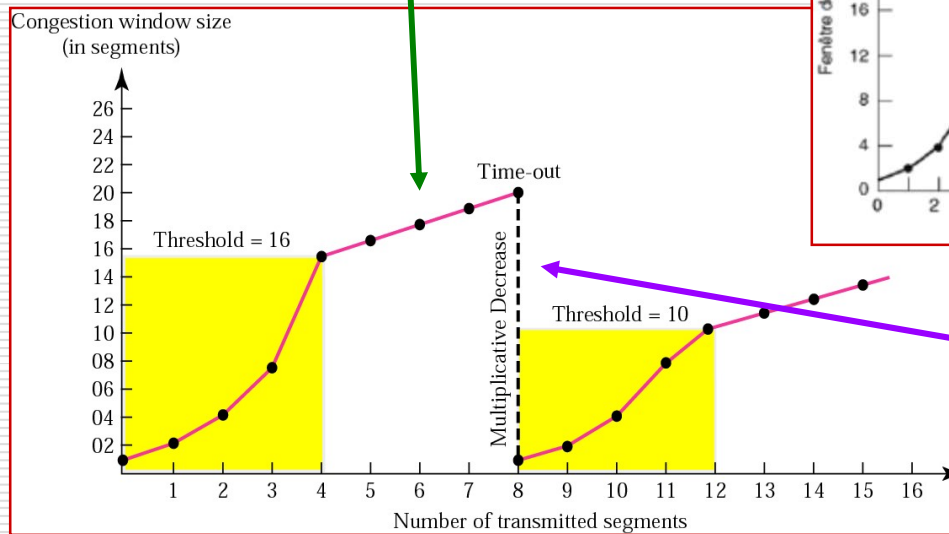
A chaque ACK reçu :
if CWND < SSTresh then
    CWND ← CWND + 1;
else
    CWND ← CWND + 1/CWND;

if retransmission constatée then
    SSTresh ← Max(CWND DIV 2, 2); // SSTresh tjrs ≥ 2
    CWND ← 1
```

TCP (18/31) Contrôle de congestion (11/23)

□ TCP Tahoe (SS+CA) :

AI : Additive Increase

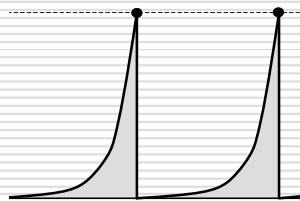


MD : Multiplicative Decrease

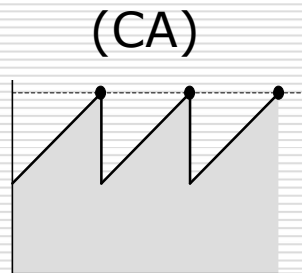
TCP (19/31) Contrôle de congestion (12/23)

□ TCP Tahoe (SS+CA) :

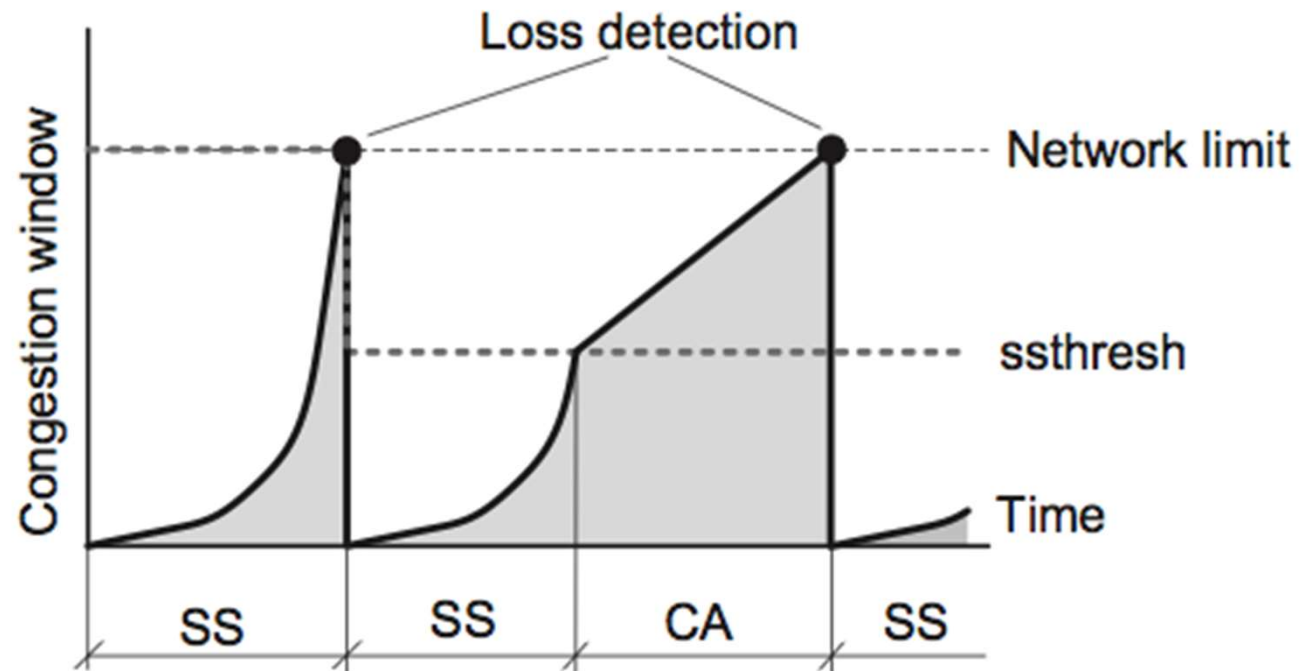
- Illustration efficacité SS+CA : combinaison
 - découverte rapide ressource réseau (SS)
 - efficacité à long terme (CA)



(SS)



(CA)



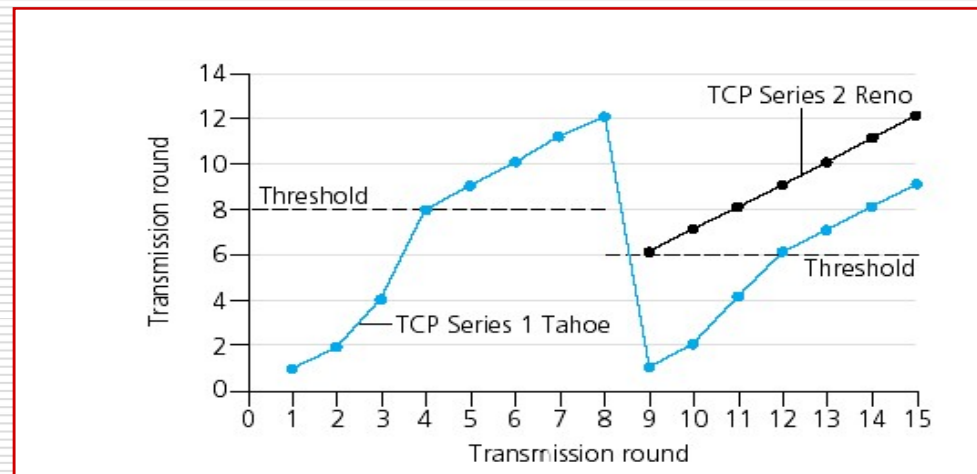
TCP (20/31) Contrôle de congestion (13/23)

□ TCP Reno :

- Tahoe inefficace dans le cas de pertes isolées
== > *atténuer la MD dans ces cas là*
- Algorithme :

```
if RTO expire then  
  SSTresh  $\leftarrow$  Max(CWND DIV 2, 2);  
  CWND  $\leftarrow$  SSTresh; // on ne redémarre pas le SS à 1
```

- Illustration :



TCP (21/31) Contrôle de congestion (14/23)

□ TCP Reno :

□ Fast Recovery (recouvrement rapide) (pas obligatoire):

- Si 3 acquittements dupliqués reçus (on reçoit du trafic, donc pas de forte congestion du réseau), réduire la fenêtre de moitié
- Combiner cette MD atténuée avec un **Fast Retransmit** et laisser l'émetteur en CA-like
- Cette phase stoppe à l'arrivée de l'ACK = $x+N$
- Puis passage à la phase CA

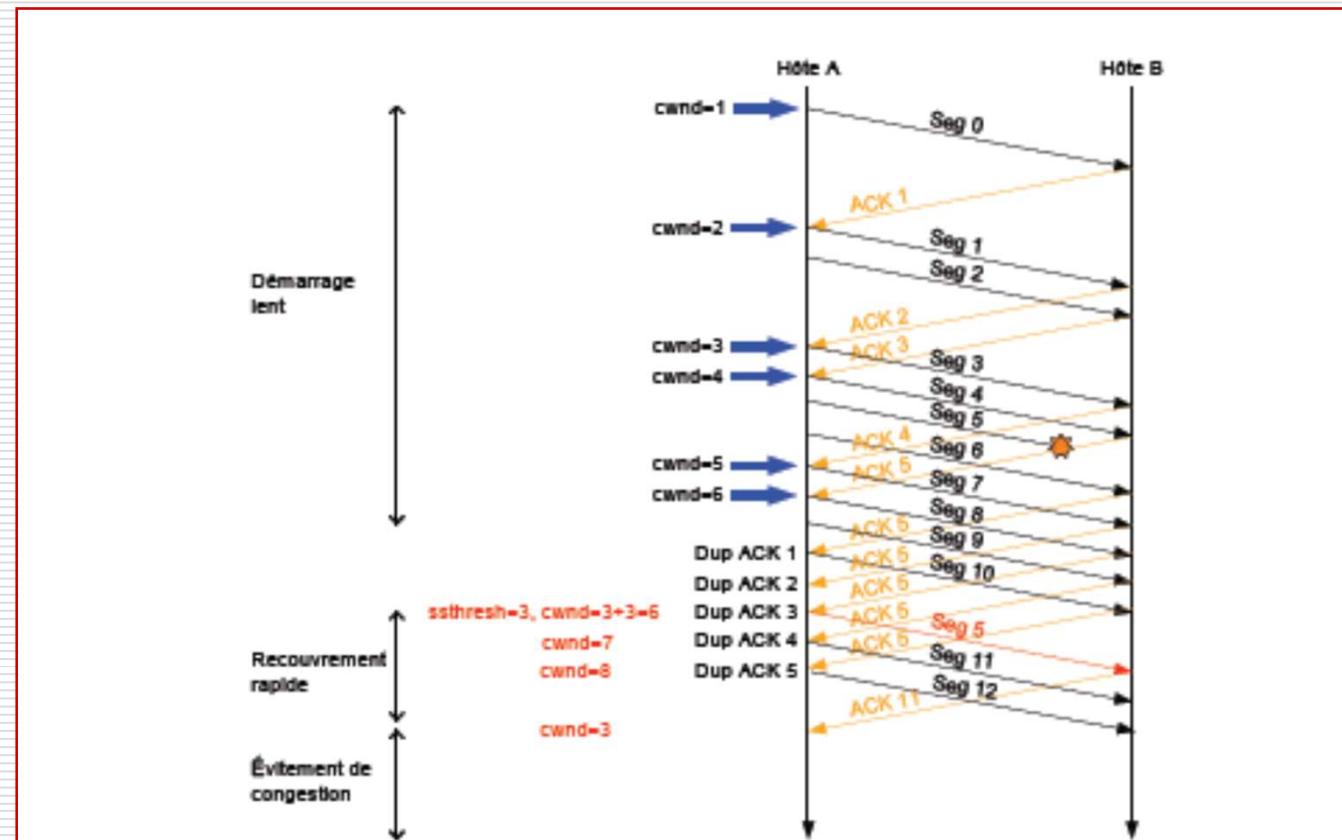
□ Algorithme :

```
If Trois ACK = x reçus then  
    SEND-AGAIN(segment); // Fast Retransmit  
    // Multiplicative Decrease  
    SSTresh  $\leftarrow$  Max(CWND DIV 2, 2);  
    CWND  $\leftarrow$  SSTresh + 3; // window inflating  
If nouveau ACK = x reçu then  
    // Additive Increasing  
    CWND  $\leftarrow$  CWND + 1;  
    // Arrêt Fast Recovery  
If ACK = x+n reçu then  
    CWND  $\leftarrow$  SSTresh; // window deflating
```

TCP (22/31) Contrôle de congestion (15/23)

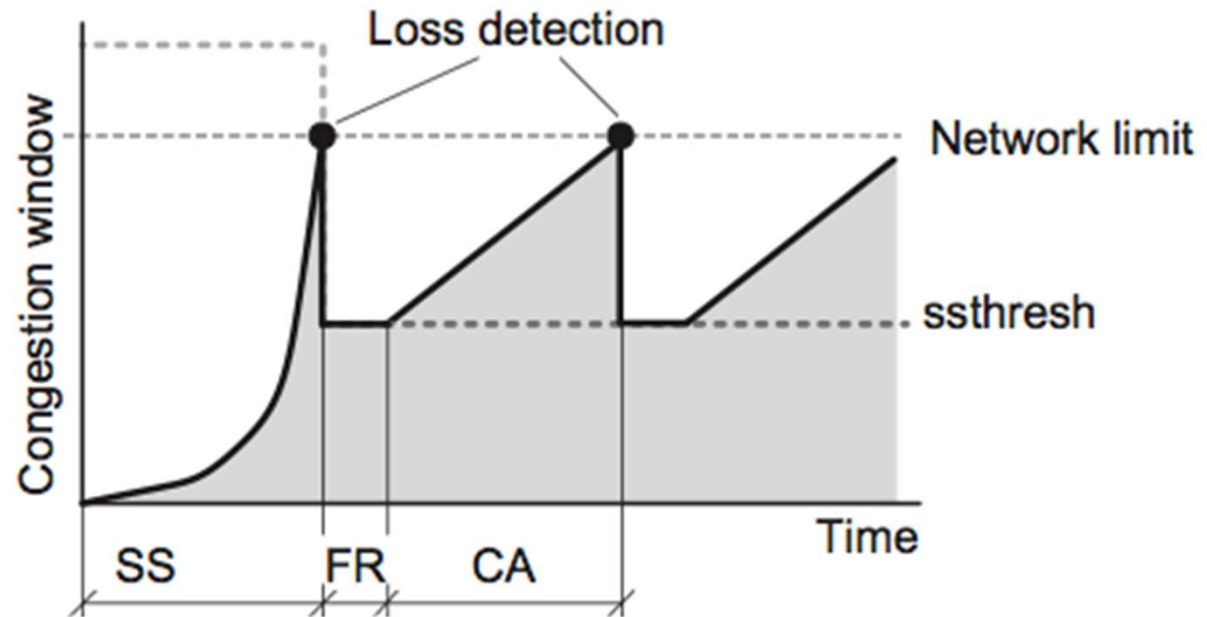
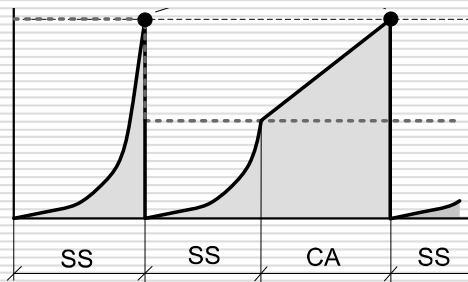
□ TCP Reno :

■ Illustration :



TCP (23/31) Contrôle de congestion (16/23)

□ Tahoe vs Reno :



Stratégie :

si 3 ACK = x reçus == > congestion non explicite (FR puis CA)

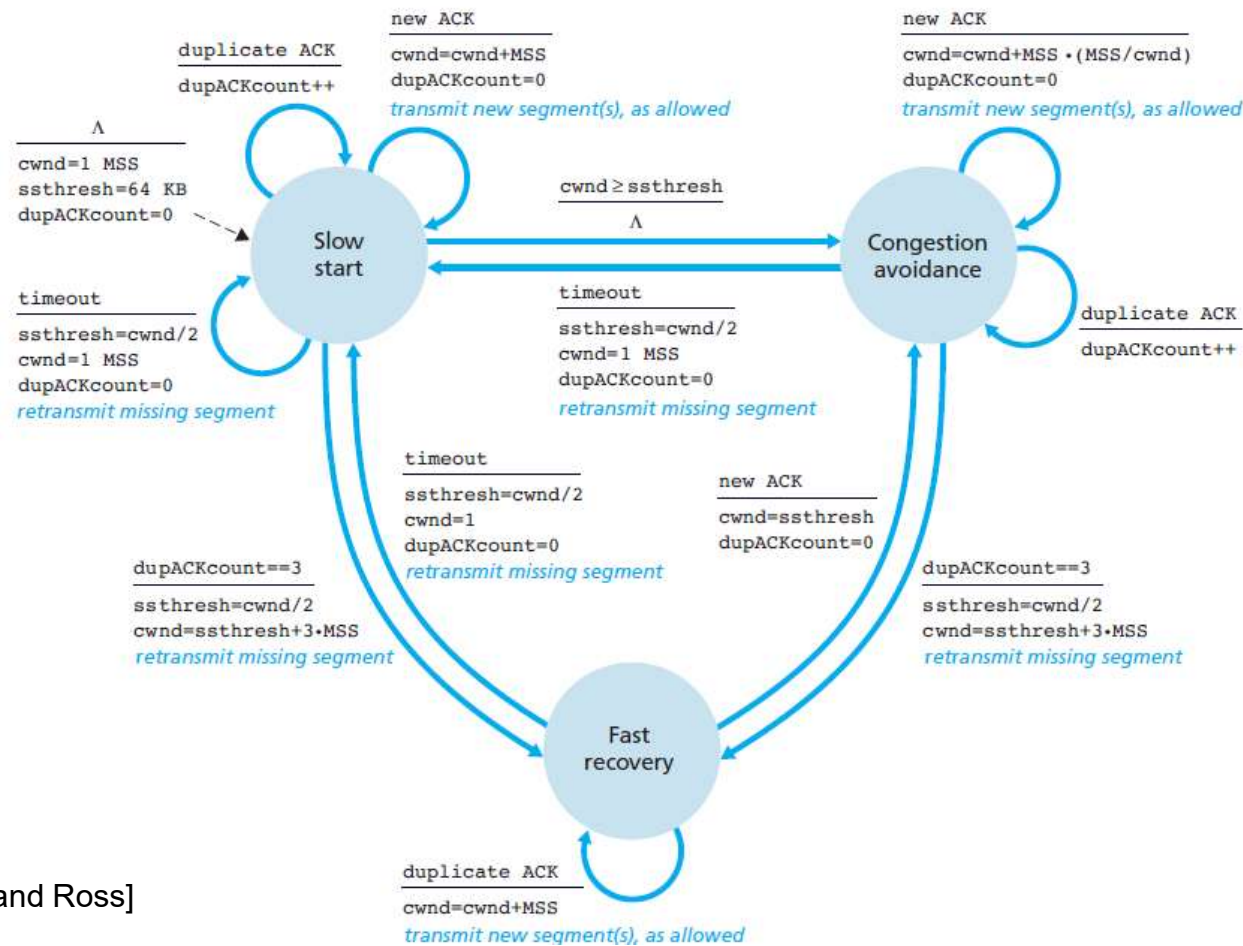
si timer expire == > congestion plus probable (SS puis CA)

== > TCP agressif et réactif

(les autres doivent être TCP-friendly mais si nécessaire...)

TCP (24/31) Contrôle de congestion (17/23)

□ FSM du CC de TCP Reno



[Fig. Kurose and Ross]

TCP (25/31) Contrôle de congestion (18/23)

□ TCP New Reno :

- Reno moins performant que Tahoe dans le cas de forte saturation et de pertes multiples (trous)
= = > timers déclenchés !
= = > transmission ralentie
- Gestion d'acquittements partiels :
permettre de retransmettre d'autres segments avant l'expiration de leur délai de non réception d'acquittement
= = > rester dans la phase FR jusqu'à ce que les segments déjà transmis mais non encore acquittés au moment où TCP est entré en FR soient acquittés
- En résumé : Amélioration du FR de Reno lors de pertes multiples détectées
- L'un des plus implémentés

TCP (26/31) Contrôle de congestion (19/23)

□ TCP SACK :

- Bénéficier de la possibilité d'[acquittements sélectifs](#) transférés dans le champ options de l'en-tête d'un segment TCP [RFC 2018].
- L'émetteur peut alors recouvrir plus rapidement les pertes multiples et les correctes.
(diminuer les retransmissions parasites – spurious).
- Meilleures performances car la probabilité de pertes de plusieurs segments est élevée en particulier dans le cas de réseaux où la fenêtre d'émission a une taille importante
- Illustration :
 - Les segments 5 à 25 ont été transmis, les 5, 12 et 18 sont perdus
 - Le récepteur renvoie : ACK = 5
 SACK=6-11,13-17,19-25
 - L'émetteur peut rapidement retransmettre 5, 12 et 18.
- Améliorations grâce à des options supplémentaires :
 - DACK : indiquer à l'émetteur l'occurrence de segments dupliqués
 - FACK : permet de calculer le nombre de segments en cours de transmission et non encore acquittés.

TCP (27/31) Contrôle de congestion (20/23)

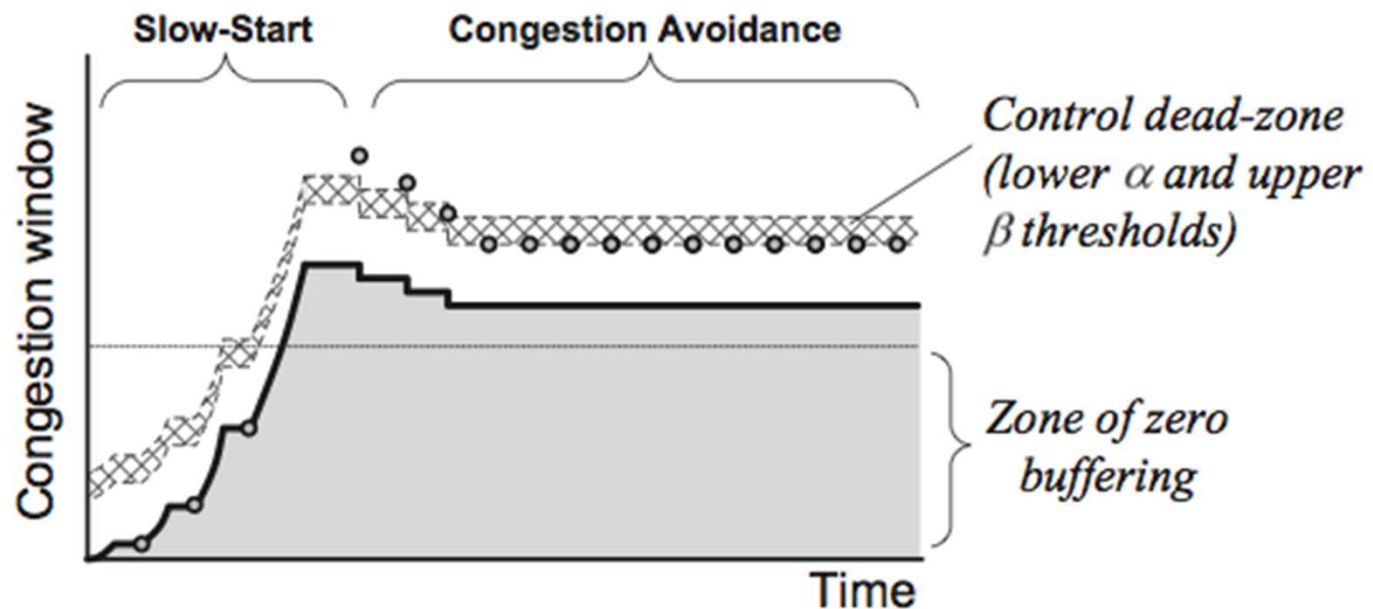
□ TCP Vegas :

- Détecter à la source des indices faisant penser qu'une congestion va survenir :
 - RTT augmente
 - Le taux d'émission est plane
- L'émetteur calcule le taux espéré d'émission (il est obtenu par division de la fenêtre de congestion par le RTT le plus petit observé jusqu'à présent)
- A chaque RTT, il calcule le taux courant et le compare au taux espéré
- Selon le résultat, variation linéaire de la fenêtre de congestion (AI/AD):
diminution ou augmentation de 1 ou de 0
- On ne se base plus sur les acquittements !
GESTION PREDICTIVE !

TCP (28/31) Contrôle de congestion (21/23)

□ TCP Vegas :

■ Illustration efficacité :

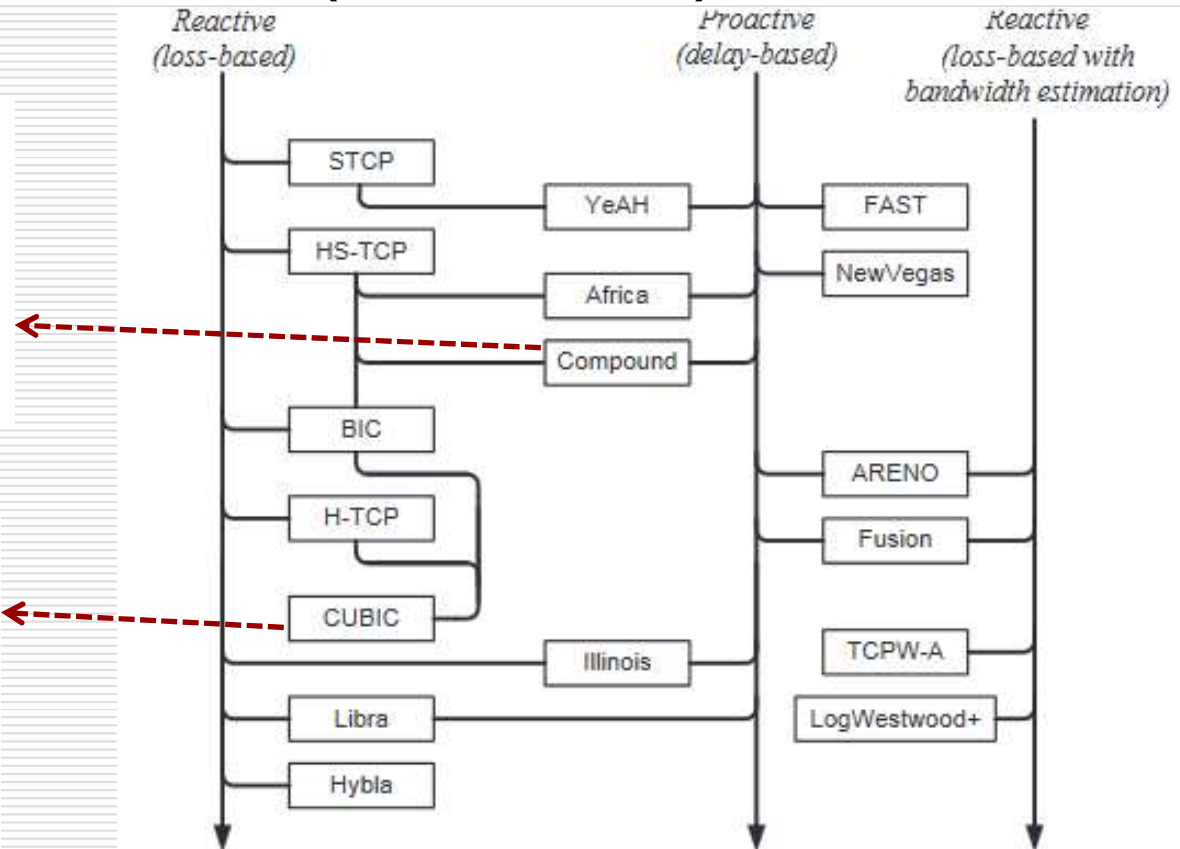
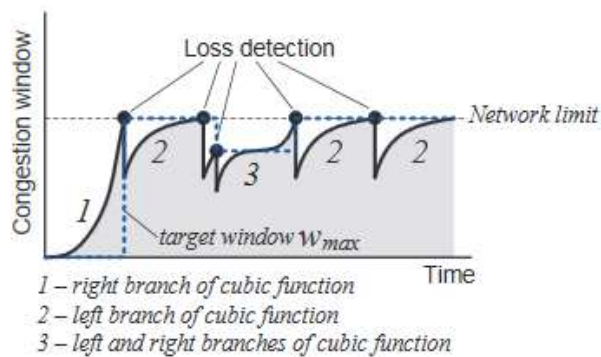
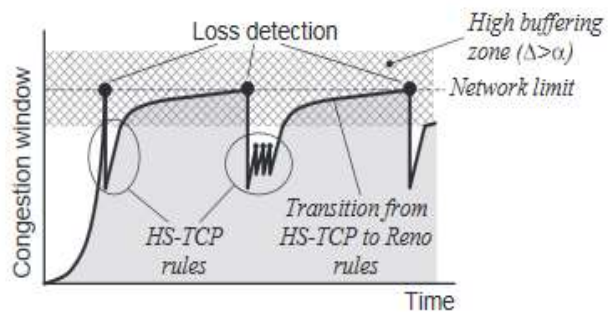


- *Expected rate calculations*
(in Slow-Start every other RTT, in Congestion Avoidance every RTT)

TCP (29/31) Contrôle de congestion (22/23)

□ Historique :

□ Nombreuses variantes (~40 en 2010) :



TCP (30/31) Contrôle de congestion (23/23)

□ Stratégies avec participation des routeurs

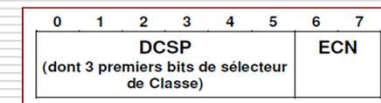
- Les routeurs participent activement au contrôle de congestion
- TCP est averti explicitement ou pas

■ RED (Random Early Detection)

- Principe de base :
 - Le routeur mesure la longueur moyenne des files d'attente
 - Dès qu'une congestion s'annonce, RED supprime de façon aléatoire des paquets et avertit par-là même TCP
 - Le taux d'élimination est fonction de la longueur de la file d'attente
- Extension : Weighted RED (plusieurs files caractéristiques différentes)

■ ECN (Explicit Congestion Notification) [RFC 3168]

- Principe :
 - Le routeur marque les paquets IP (bit CE)
 - A la réception TCP, FR/SS, puis positionne le bit CWR-Echo à chaque ACK pour provoquer une réduction de fenêtre chez l'émetteur du paquet marqué CE.
 - Celui-ci enverra un bit CWR positionné lorsqu'il a réduit sa fenêtre



TCP (31/31) Références (1/1)

□ Principales RFCs :

- RFC 791
- RFC 2001
- RFC 2018
- RFC 2581
- RFC 2582
- RFC 3168

□ Contrôle de congestion :

- *Host to Host Congestion Control for TCP*
A. Afanasyev *and al.*
In IEEE Communications Surveys and Tutorials Vol 12. 2010
- *Internet Control Congestion*
Subir Varma
Ed. Morgan Kaufmann 2015

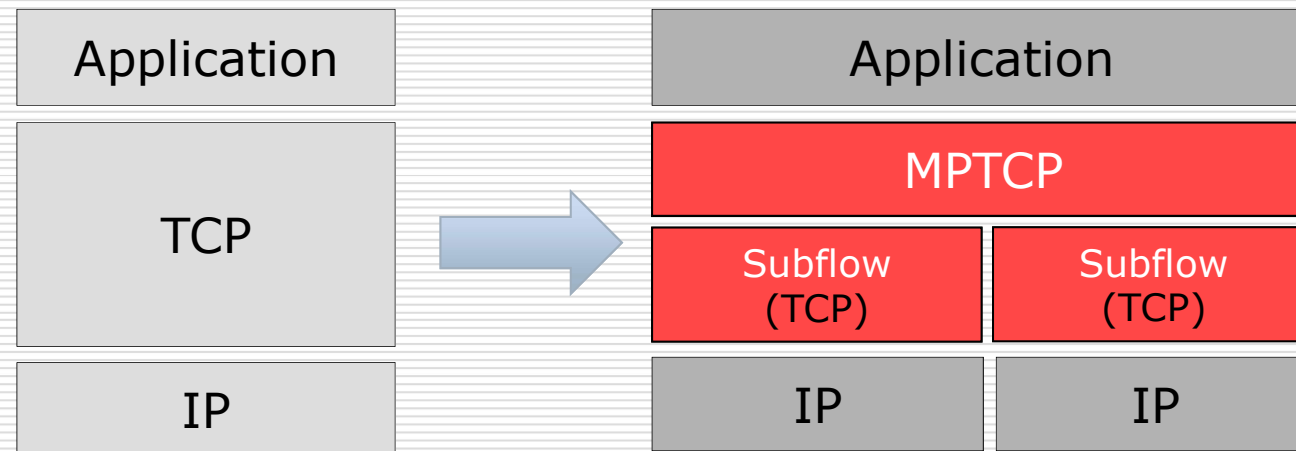
MPTCP (1/11) Généralités (1/3)

- ❑ MPTCP est une extension évolutive de TCP
- ❑ Aucune incidence sur l'existant (Application, Réseau)
- ❑ MPTCP : **Multipath TCP** [RFC 8684] *Sdt Tracks 03/20*
« TCP extensions for Multipath Operation with Multiple Adresses »

- ❑ Option TCP numéro 30
 - Plusieurs sous-types :
 - ❑ MP_CAPABLE (0x0)
 - ❑ MP_JOIN (0x1)
 - ❑ DSS (0x2)
 - ❑ ADD_ADDR (0x3)
 - ❑ REMOVE_ADDR (0x4)
 - ❑ MP_PRIO (0x5)
 - ❑ MP_FAIL (0x6)
 - ❑ MP_FASTCLOSE (0x7)

MPTCP (2/11) Généralités (2/3)

□ Architecture protocolaire



- Une connexion MPTCP fournit une connexion orientée octets bidirectionnelle entre deux processus d'application
- Cependant MPTCP permet aux hosts d'utiliser différents chemins avec des adresses IP différentes pour échanger des paquets de la connexion MPTCP
- Pour la couche IP, chaque *subflow* apparaît comme un flux TCP classique mais dont les segments sont porteurs d'un nouveau type d'option.
- MPTCP gère dynamiquement la création, la suppression et l'utilisation de ces subflows dont le nombre peut fluctuer durant la durée de vie de la connexion MPTCP.

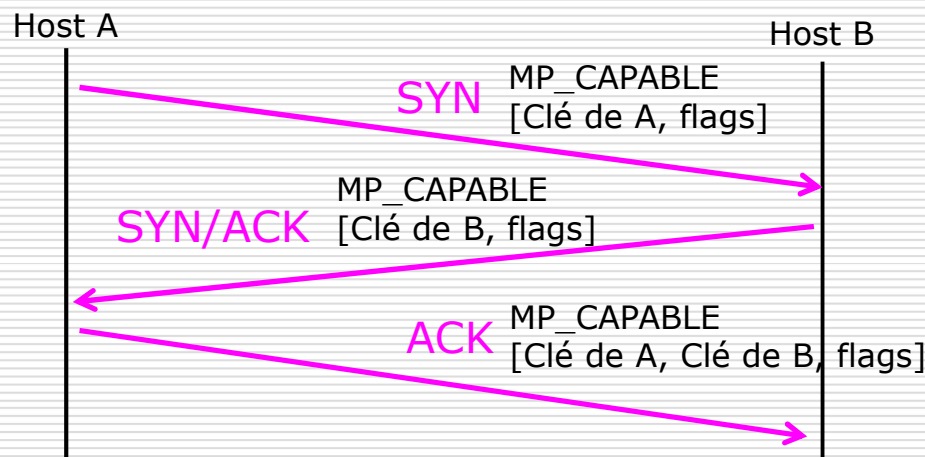
MPTCP (3/11) Généralités (3/3)

□ MPTCP : terminologie

- **Path** : séquence de liens entre un émetteur et un récepteur, défini par un quadruplet de deux paires (@sse IP, #port).
- **Subflow** : un flux de segments TCP, assimilable à une connexion TCP classique opérant sur un *path* individuel et appartenant à une connexion MPTCP.
- **Connexion MPTCP** : associée à une seule socket, il s'agit d'un ensemble d'un ou plusieurs *subflows* sur lequel une application peut communiquer entre deux hosts.
- **Data-level** : données utiles à transférer sur une connexion, qui sont transportées via des *subflows*.
- **Token** : un identifiant unique de connexion MPTCP attribué localement par un host à une connexion MPTCP.
- **Host** : un équipement d'extrémité implémentant MPTCP et éventuellement acceptant des connexions MPTCP.

MPTCP (4/11) Fonctionnement (1/8)

- Etablissement d'une connexion MPTCP :
 - Suit la procédure en 3 phases de l'établissement de connexion TCP SYN, SYN/ACK, ACK
 - Mais chacun des 3 segments utilise obligatoirement l'option **MP_CAPABLE** pour :
 - Vérifier si l'hôte distant supporte MPTCP
 - Echanger des informations quant à l'authentification nécessaire à l'établissement des *subflows*
 - Si problème : retour à une connexion TCP unique



flags :

A : Checksum requis

B : *utilisation future*

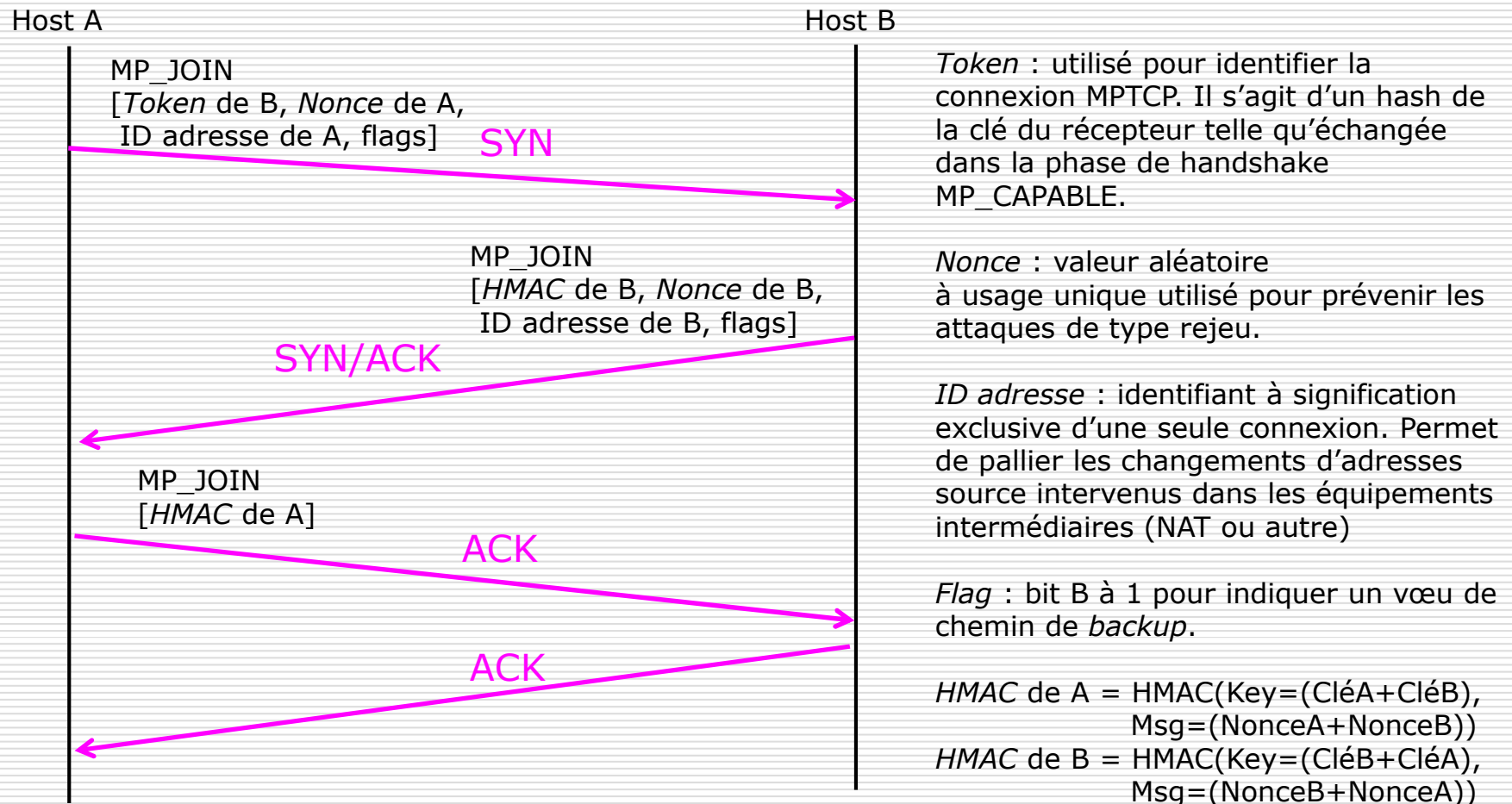
C à H : utiles pour la négociation d'algorithmes de cryptographie. Pour l'instant seul le bit H indique l'usage de codes HMAC-SHA1.

MPTCP (5/11) Fonctionnement (2/8)

- Associer un nouveau *subflow* à une connexion MPTCP existante :
 - Un host peut initier un nouveau *subflow* sur une paire d'adresses actuellement non utilisées.
 - Il est plutôt attendu que ce soit l'initiateur de la connexion qui initie également les subflows.
 - Chaque nouveau *subflow* s'établit aussi selon la procédure en 3 phases de l'établissement de connexion TCP; Mais chacun des 3 segments utilise l'option **MP_JOIN**.
 - Un jeton généré à partir de la clé est utilisé pour identifier à quelle connexion MPTCP le *subflow* est rattaché. Un code d'authentification est également échangé (HMAC – *Hash-based Message Authentication*). Enfin, un ID d'adresse est également présent pour référencer l'adresse source même si celle-ci subit une translation.
 - Si problème : envoi d'un RST.

MPTCP (6/11) Fonctionnement (3/8)

□ Illustration de l'initiation d'un subflow :



MPTCP (7/11) Fonctionnement (4/8)

□ Transfert de données :

- MPTCP doit garantir un transport fiable malgré l'apparition ou disparition possible de *subflows*.
- Le **DSN** (*Data Sequence Number*) est un numéro de séquence sur 8 octets pour numéroté toutes les données d'une connexion MPTCP. Le numéro initial (IDSN) est généré à partir de la clé de l'équipement suite à l'établissement de la connexion MPTCP.
- Chaque *subflow* utilise, de façon classique, son propre numéro de séquence sur 4 octets.
- L'option MCTCP **DATA_SEQUENCE_SIGNAL** établit la correspondance entre l'espace de numérotation du *subflow* et celui des données de la connexion MCTCP.
- Ainsi, le **DSM** (*Data Sequence Mapping*) transporté dans cette option, comprend le numéro de séquence du *subflow*, le **DSN**, et la longueur pour laquelle cette correspondance est valide. Un acquittement de niveau connexion peut y être également transporté (**Data Ack**).
- Ainsi, deux niveaux d'acquittement sont gérés en MPTCP : au niveau *subflow*, et au niveau connexion MPTCP pour gérer l'avancement de l'échange des flux d'octets puisque tous les *subflows* partagent les mêmes *buffers* d'émission et de réception.

MPTCP (8/11) Fonctionnement (5/8)

□ Option DDS (Data Sequence Signal) :

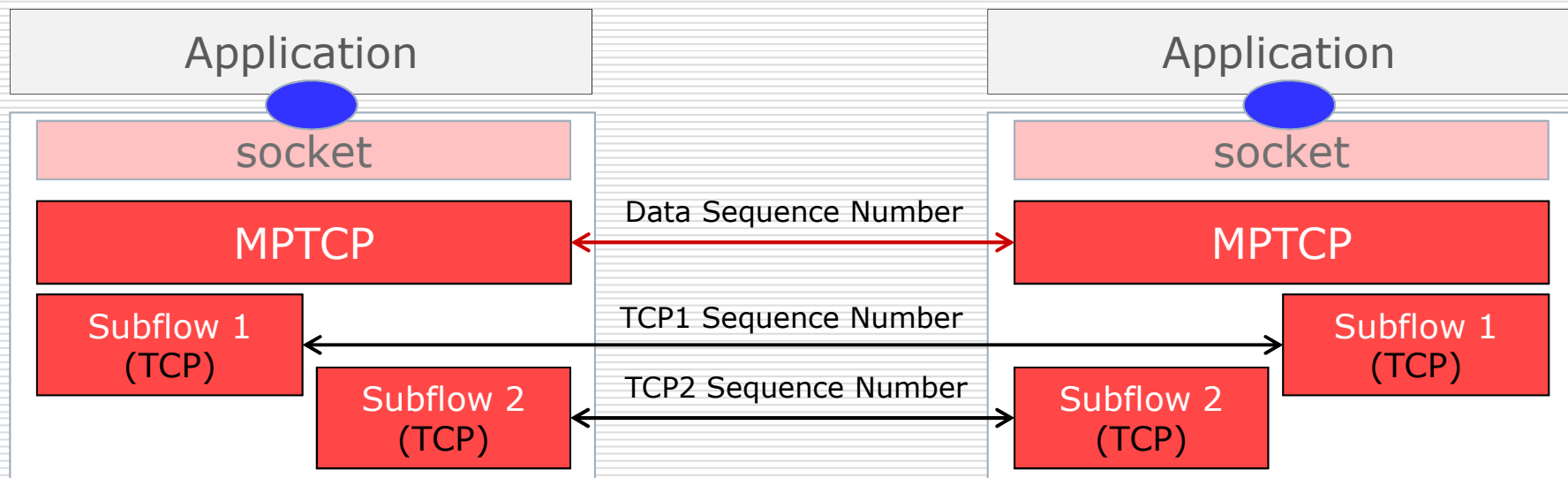
- Respect du format des options de TCP : Code (**30**), Longueur
- Le champ sous-type, sur 4 bits, spécifie le type de message MPTCP. Chaque type de message est enregistré par l'IANA.
- Enfin suivent les données spécifiques au type de message MPTCP.

30	Longueur	0x2	Flags (5 bits)
Data ACK (4 ou 8 octets)			
Data Sequence Number (DSN) (4 ou 8 octets)			
Subflow Sequence Number (4 octets)			
Data-Level Length (2 octets)		Checksum (2 octets)	

flags : A = Data ACK présent
 a = Data ACK sur 8 octets, 4 octets sinon
 M = DSN, SSN, Length et Checksum présents
 m = DSN sur 8 octets, 4 octets
 F = indique un DATA_FIN (fin des données pour l'émetteur)

MPTCP (9/11) Fonctionnement (6/8)

□ Synthèse :



MPTCP (10/11) Fonctionnement (7/8)

- Informer l'autre host d'une autre adresse potentielle :
 - L'ensemble des adresses IP associées à un équipement multi-domicilié peut évoluer durant la vie de la connexion MPTCP. Si un host veut ouvrir un deuxième *subflow* à partir d'une nouvelle adresse, il initie son établissement et informe implicitement l'autre équipement de cette nouvelle adresse.
 - Parfois, un équipement veut avertir de la disponibilité d'une adresse mais sans établir un nouveau *subflow* : le premier envoie alors une option **ADD_ADR** au second
 - Il existe l'option duale **REMOVE_ADR** pour indiquer la non disponibilité d'une adresse
 - Dans les deux cas l'ID d'adresse est mentionné.

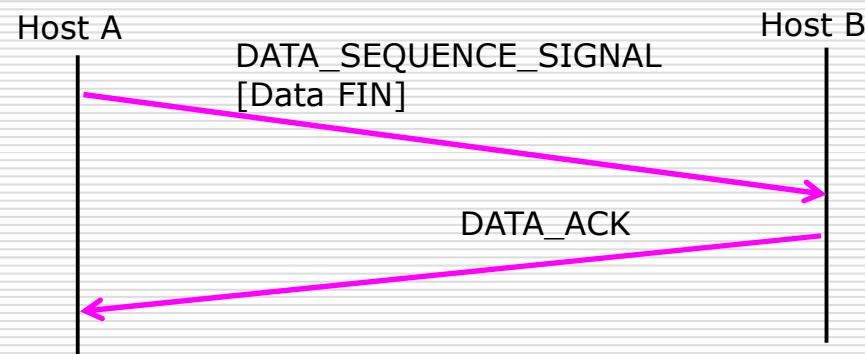
MPTCP (11/11) Fonctionnement (8/8)

□ Changer la priorité d'un chemin :

- Les équipements peuvent indiquer, lors de son initialisation, s'ils souhaitent qu'un subflow soit utilisé normalement ou en tant que chemin de recours.
- Durant la connexion, un équipement peut demander le changement de cette priorité grâce à l'option MCTCP **MP_PRIO**.

□ Terminer une connexion MPTCP

- Lorsqu'un host veut informer le second qu'il n'a plus de données à émettre, il le signale par un **Data Fin** dans l'option Data_Sequence_Signal (même signification qu'un bit FIN).
- Une fois toutes les données de la connexion MPTCP bien reçues, ce message est acquitté au niveau connexion grâce à l'option MPTCP **DATA_ACK**.



SCTP (1/14) Généralités (1/1)

- ❑ SCTP une combinaison améliorée d'UDP et TCP
- ❑ Caractéristiques générales :
 - Mode connecté, notion d'associations full-duplex
 - La fiabilité des échanges des données est dissociée de la délivrance de celles-ci
 - = = > *SCTP permet de s'adapter à des besoins différents (ex : seulement la fiabilité et peu importe l'ordre).*
 - = = > *arrival-order delivery*
 - Orienté multi-flots : les données sont séparées et transférées en plusieurs flots indépendants
 - = = > *une perte n'entrave qu'un des flots en terme de retransmission*
 - Transfert fiable : contrôle d'échange
 - = = > *contrôle de la validité des données (checksum, numérotation et acquittement, retransmission)*
 - Multi-domiciliation : plusieurs adresses IP peuvent être associées à chacune des extrémités d'une association SCTP
 - = = > *augmenter la tolérance aux pannes de réseau.*
 - Contrôle de congestion et contrôle de flux identiques à TCP
 - Protocole extensible

SCTP (2/14) Gestion des associations (1/6)

- Point d'extrémité SCTP (**Endpoint**)
 - Il est associé à un seul processus local communicant
 - Matérialisé par une socket
 - Il est identifié par un numéro de port et une liste non vide d'adresse IP du nœud local
 - Remarque : dans le cas d'une multi-domiciliation, deux applications peuvent utiliser le même numéro de port SCTP à condition que les adresses IP associées à leur points d'extrémité respectifs soient utilisées de façon exclusive.

SCTP (3/14) Gestion des associations (2/6)

□ Association SCTP

- Une association est établie entre un point d'extrémité local et un point d'extrémité distant

- Exemple d'identification :

ass1 = { [10.1.61.11 : 2223], [161.10.8.221, 120.1.1.5 : 80] }

- Remarques :

- Un même point d'extrémité local peut supporter des associations multiples (communications 1-N - différent du multicast)

= = > un host peut dialoguer avec plusieurs autres à partir d'une socket unique

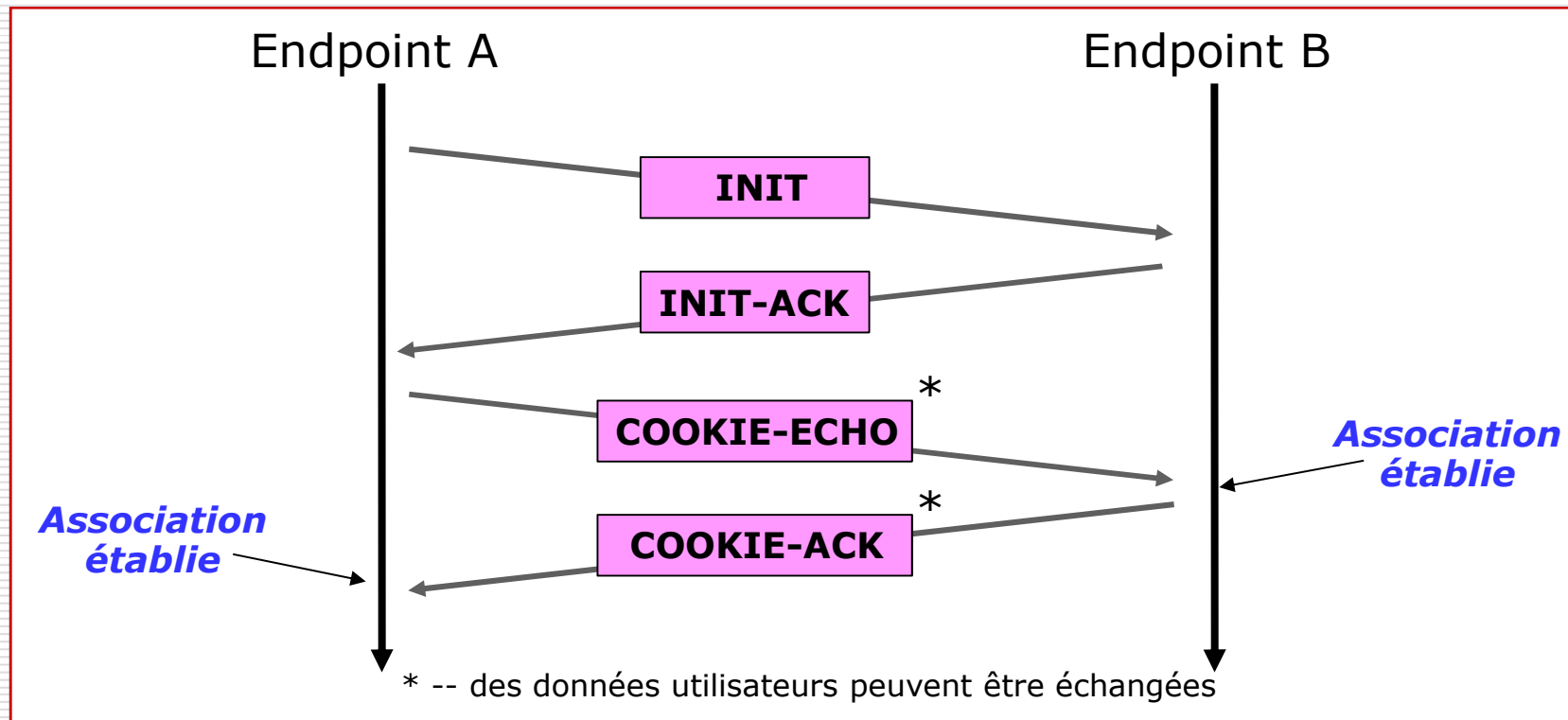
- Une association a exactement deux points d'extrémités

ass2 = { [10.1.65.14 : 4586], [161.10.8.221, 120.1.1.5 : 80] }

SCTP (4/14) Gestion des associations (3/6)

□ Etablissement

- La procédure s'opère en quatre phases d'échanges de messages (chunk) spécifiques :



SCTP (5/14) Gestion des associations (4/6)

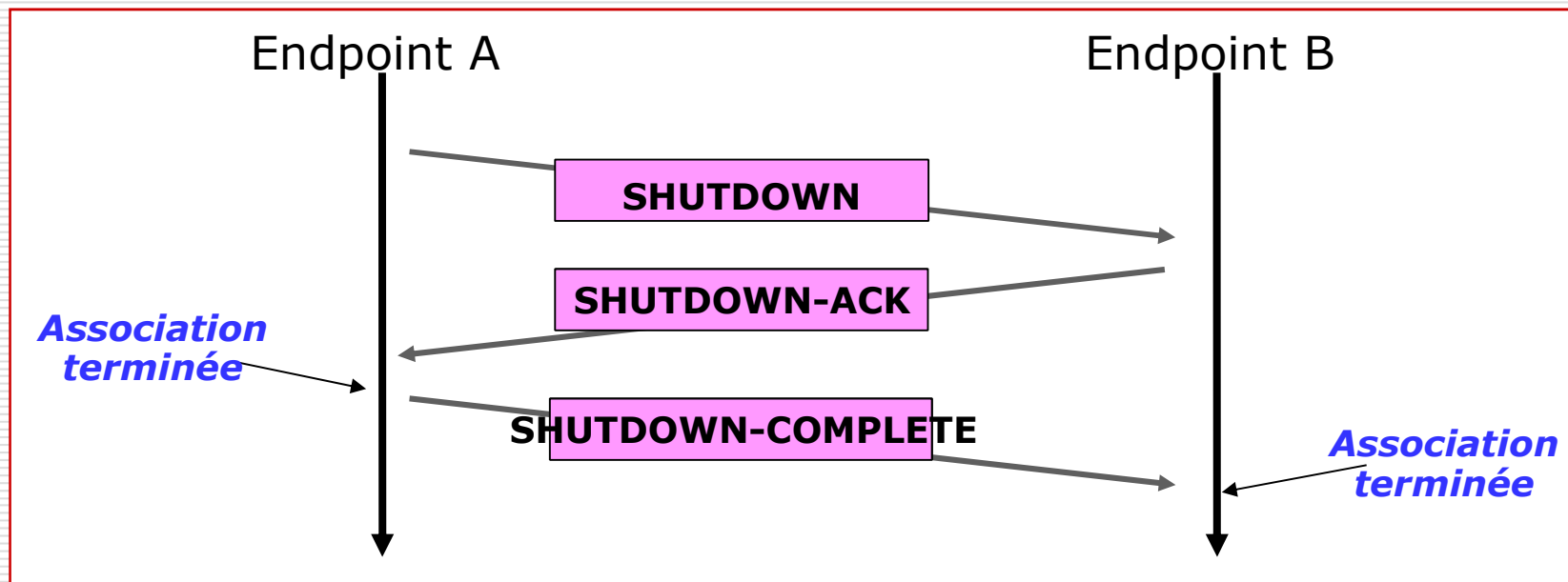
□ Etablissement

- Constitution **INIT** :
 - Préciser les adresses IPv4 et IPv6
 - Générer et inclure
 - un VT (Verification Tag)
 - un ITSN (Initial Transport Sequence Number)
 - Armer un temporisateur d'attente d'acquittement
- Constitution **INIT-ACK**
 - Même paramètres qu'au dessus
 - Un cookie signé (MD ou SHA-1) estampillé
 - Aucun état n'est créé pour cette association à cette étape
- Constitution **COOKIE-ECHO**
 - Cookie reçu (deux octets modifiés)
 - Éventuellement des données
- Constitution **COOKIE-ACK**
 - Après vérification et création d'un TCB, ce shunk est constitué puis envoyé (avec éventuellement des shunks de données)

SCTP (6/14) Gestion des associations (5/6)

□ Terminaison

- La procédure élégante s'opère en trois échanges :

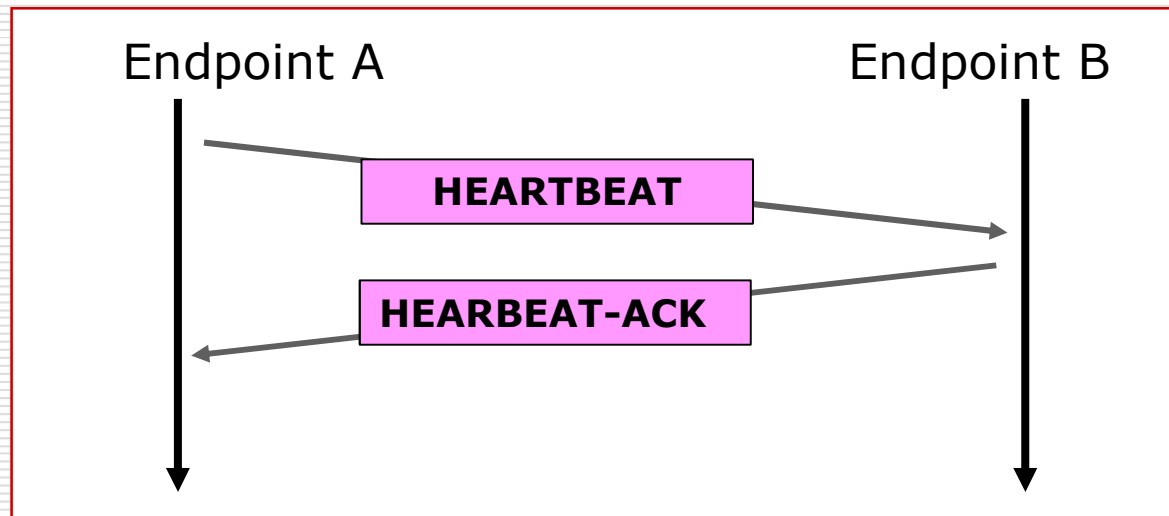


- SCTP ne supporte pas des terminaisons unilatérales
- Possibilité de terminaison abrupte : chunk ABORT

SCTP (7/14) Gestion des associations (6/6)

□ Surveillance connectivité

- Procédure périodique de test de connectivité :



- Aide au calcul du RTT

SCTP (8/14) Transfert de données (1/3)

- Préparation à l'envoi :
 - Selon la taille du message appartenant à un flot :
 - Chunk DATA unique :
 - Taille du message inférieure à la taille maximale d'un chunk (MCS, en principe calculée à partir de la plus petite MTU des adresses possibles du endpoint destinataire)
 - Un TSN, un Identifiant de flot (SID) et aussi un SSN (Stream Sequence Number) si ordre requis, sont assignés à ce chunk
 - Fragmentation en plusieurs chunks DATA :
 - Tous les fragments utilisent les mêmes SID et SSN
 - Chaque fragment porte un TSN unique et des bits de positionnement du fragment (premier, dernier, milieu)
- Réception et délivrance des données :
 - Réassemblage des fragments en s'appuyant sur les bits et les TSNs
 - Remise à l'application du message reçu (immédiatement si flot non ordonné, après re-ordonnancement sinon)

SCTP (9/14) Transfert de données (2/3)

□ Acquittements sélectifs :

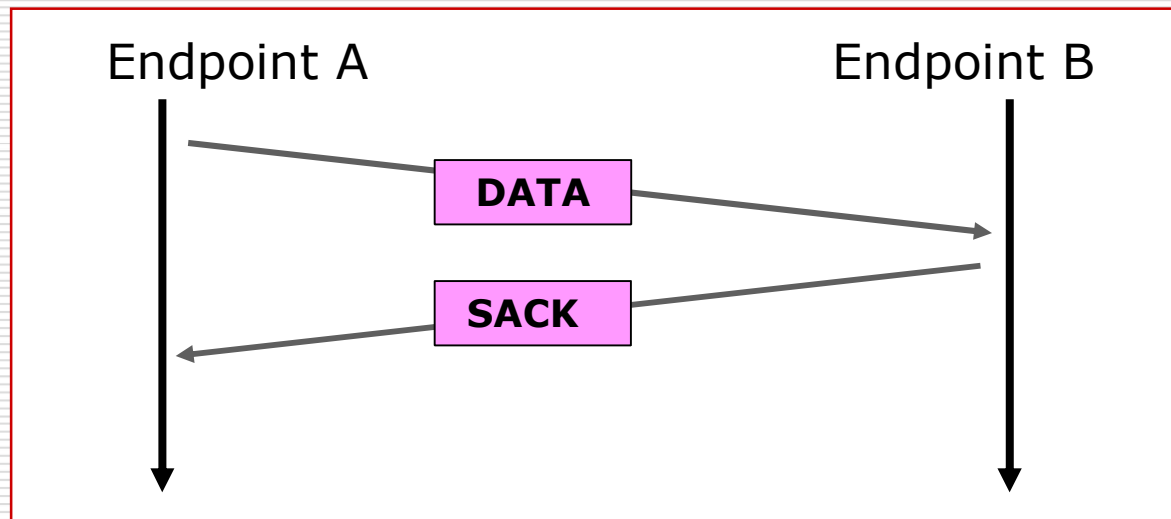
- Mécanismes d'acquittements sélectifs avec temporisation
- L'acquittement cumulatif porte sur le TSN
- Les segments « Hors séquence » reçus sont acquittés dans la partie sélective (idem TCP SACK)
- Un SACK peut également indiquer des données dupliquées

□ Gestion des retransmissions :

- Valeurs calculées sur RTT et RTO de façon similaire à TCP
- Mécanisme de Retransmission Rapide (FR) identique à TCP

SCTP (10/14) Transfert des données (3/3)

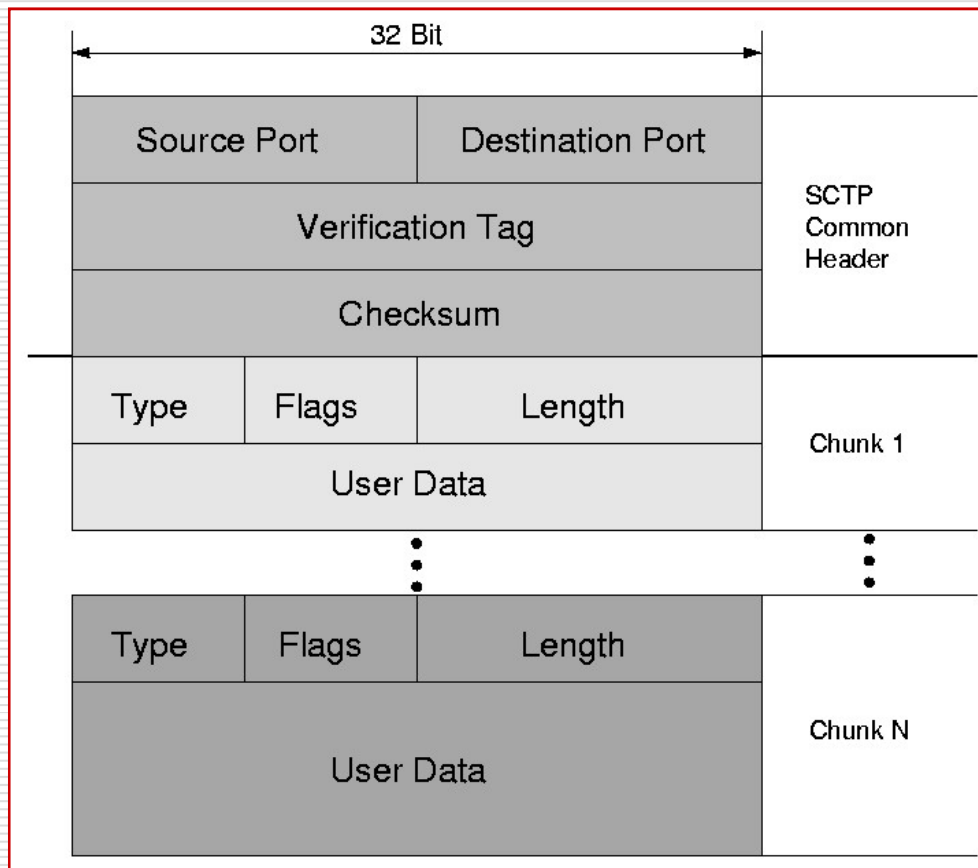
□ Illustration :



- Mécanisme de contrôle de flux
- Mécanisme de contrôle de congestion (AIMD)

SCTP (11/14) Protocole (1/3)

□ Format d'un segment :



SCTP (12/14) Protocole (2/3)

□ Liste des types de chunks de base :

ID	Value	Chunk Type
-----	-----	
0	-	Payload Data (DATA)
1	-	Initiation (INIT)
2	-	Initiation Acknowledgement (INIT ACK)
3	-	Selective Acknowledgement (SACK)
4	-	Heartbeat Request (HEARTBEAT)
5	-	Heartbeat Acknowledgement (HEARTBEAT ACK)
6	-	Abort (ABORT)
7	-	Shutdown (SHUTDOWN)
8	-	Shutdown Acknowledgement (SHUTDOWN ACK)
9	-	Operation Error (ERROR)
10	-	State Cookie (COOKIE ECHO)
11	-	Cookie Acknowledgement (COOKIE ACK)
12	-	Reserved for Explicit Congestion Notification Echo (ECNE)
13	-	Reserved for Congestion Window Reduced (CWR)
14	-	Shutdown Complete (SHUTDOWN COMPLETE)
15 to 62, 64 to 126, 128 to 190, 192 to 254	-	reserved by IETF
63, 127, 191, 255	-	IETF-defined Chunk Extensions

SCTP (13/14) Protocole (2/3)

□ Exemples de formats de chunk :

■ DATA :

Type=0x00	Flags=UBE	Length=variable
TSN Value		
Stream Identifier		Stream Sequence Num
Payload Protocol Identifier		
Variable Length User Data		

■ SACK :

Type=3	Flags=0	Length=variable
Cumulative TSN		
Receiver window credit		
Num of Fragments=N		Num of Dup=M
Gap Ack Blk #1 start		Gap Ack Blk #1 end
Gap Ack Blk #N start		Gap Ack Blk #N end
Duplicate TSN #1		
Duplicate TSN #M		

SCTP (14/14) Les RFCs (1/1)

☐ Principales RFCs :

- ☐ RFC 2960
- ☐ RFC 3309
- ☐ RFC 3286
- ☐ RFC 2719
- ☐ RFC 3436
- ☐ RFC 4960

Bibliographie
